```
Output:
int(77)
int(7)
```

`strlen()` is very simple: you give it a string, and it tells you how many bytes are in there. It's binary safe, as the second `strlen()` call shows, and since its job is to count bytes, using it with multi-byte data won't break anything (provided you remember it is counting bytes, not characters).

```
$mbStr = "\u{1F920}";
var_dump(bin2hex($mbStr));
var_dump(strlen($mbStr));
```

```
Output:
string(8) "f09fa4a0"
int(4)
```

Unicode codepoint *0x1F920* ("Face with Cowboy Hat") is encoded with four bytes in UTF-8, which the output confirms. But what if we actually need a character count? The `mbstring` extension has us covered by providing the function `mb_strlen()`:

```
var_dump(mb_strlen($mbStr));
```

```
Output:
int(1)
```

Marvelous. We have an accurate character count of a multi-byte string. And now, because I'm in one of my moods, allow me to present this:

```
$prideFlag = "\u{1F3F3}\u{FE0F}\u{200D}\u{1F308}";
print $prideFlag . PHP_EOL;
var_dump(mb_strlen($prideFlag));
```

```
Output:
🏳️‍🌈
int(4)
```

I've thrown you a curveball. This is the Unicode Rainbow Flag character, which is multi-codepoint – remember our Bald Man (Man + ZERO_WIDTH_JOINER + Bald) from earlier? This is the same thing; it's using multiple UTF-8 characters to represent a single emoji. Kind of a pain to count character length now, huh. Let me introduce you to a new